Semordnilap

AlgoExpert · Easy · Strings

The problem

- Return pairs of distinct words where one is the reverse of the other.

Intuition

- A word and its reverse form a pair; a hash set finds each partner in constant time.

Brute force vs optimal

- Brute force: compare every pair → $O(n^2 \cdot m)$
- Optimal: hash set of words → $O(n \cdot m)$
- Remove both words once paired

Approach

- 1. Put all words in a set
- 2. For each word, check whether its reverse is present and different
- 3. Record the pair and remove both

Worked example




```
reverse('diaper') = 'repaid' ✓
pairs = [['diaper', 'repaid']]
```

'diaper' ↔ 'repaid'

Worked example




```
reverse('abc') = 'cba' ✓
pairs = [['diaper', 'repaid'], ['abc', 'cba']]
```

'abc' ↔ 'cba'

Worked example




```
reverse('test') = 'tset' ✗
```

'tset' not present

Java solution

```java
public class Semordnilap {
    public static List<List<String>> semordnilap(String[] words) {
        Set<String> set = new HashSet<>(Arrays.asList(words));
        List<List<String>> out = new ArrayList<>();
        for (String w : words) {
            String r = new StringBuilder(w).reverse().toString();
            if (!r.equals(w) && set.contains(r)) {
                out.add(List.of(w, r));
                set.remove(w);
                set.remove(r);
            }
        }
        return out;
    }
}
```

Complexity

- Time: O(n*m)
- Space: O(n*m)

Edge cases & pitfalls

- Palindromes like 'aba' don't pair with themselves
- Remove matched words to avoid duplicate pairs
- Order of output pairs

Interview tips & follow-ups

- Follow-up: count pairs, or allow duplicates

Recap

- A word and its reverse form a pair; a hash set finds each partner in constant time.
- Complexity: O(n*m)